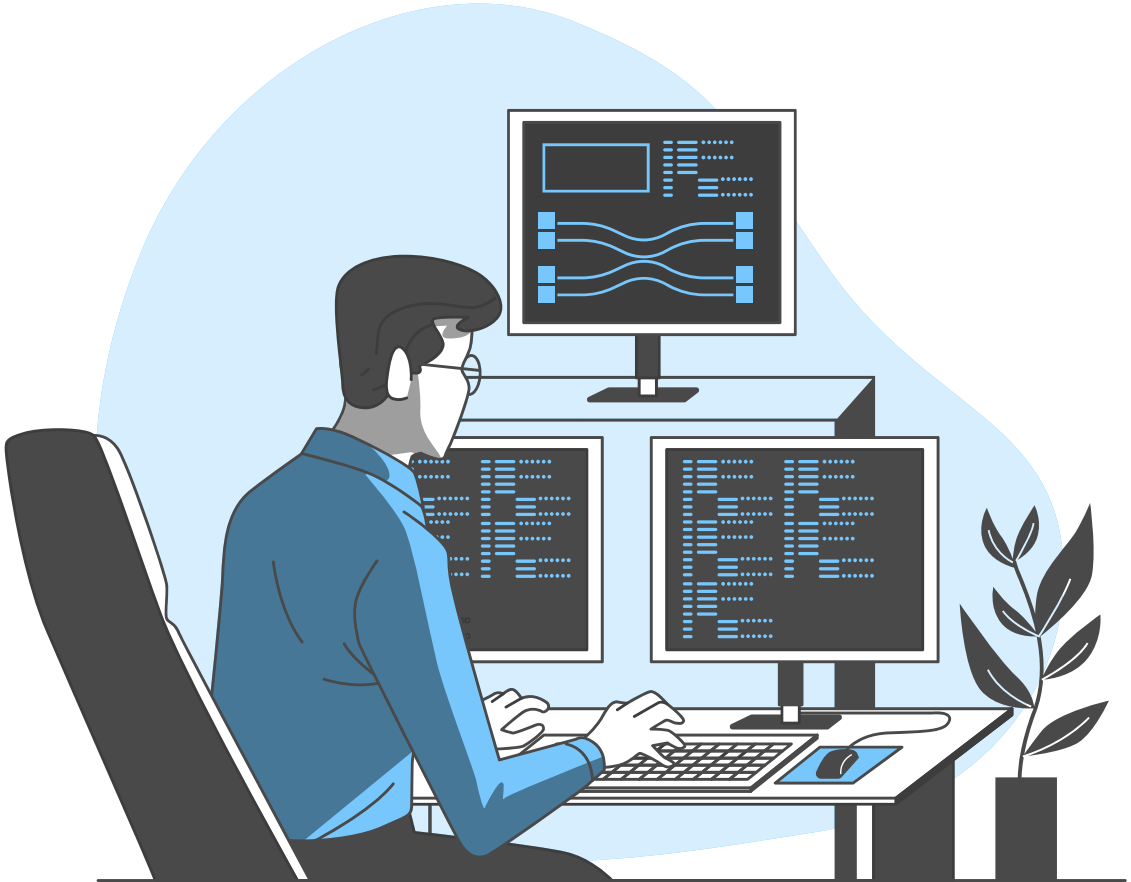


KIẾN TRÚC JAMSTACK

Phân tích hệ thống ScriptHammer
Theo mô hình 4+1 góc nhìn

Nguồn: ScriptHammer; mẫu Technology Consulting by Slidesgo

JAMstack tách giao diện dựng sẵn khỏi API

Dựng trước

Tạo HTML và tài nguyên trước khi có yêu cầu.

Phục vụ tệp

Máy chủ tĩnh hoặc CDN phân phối giao diện.

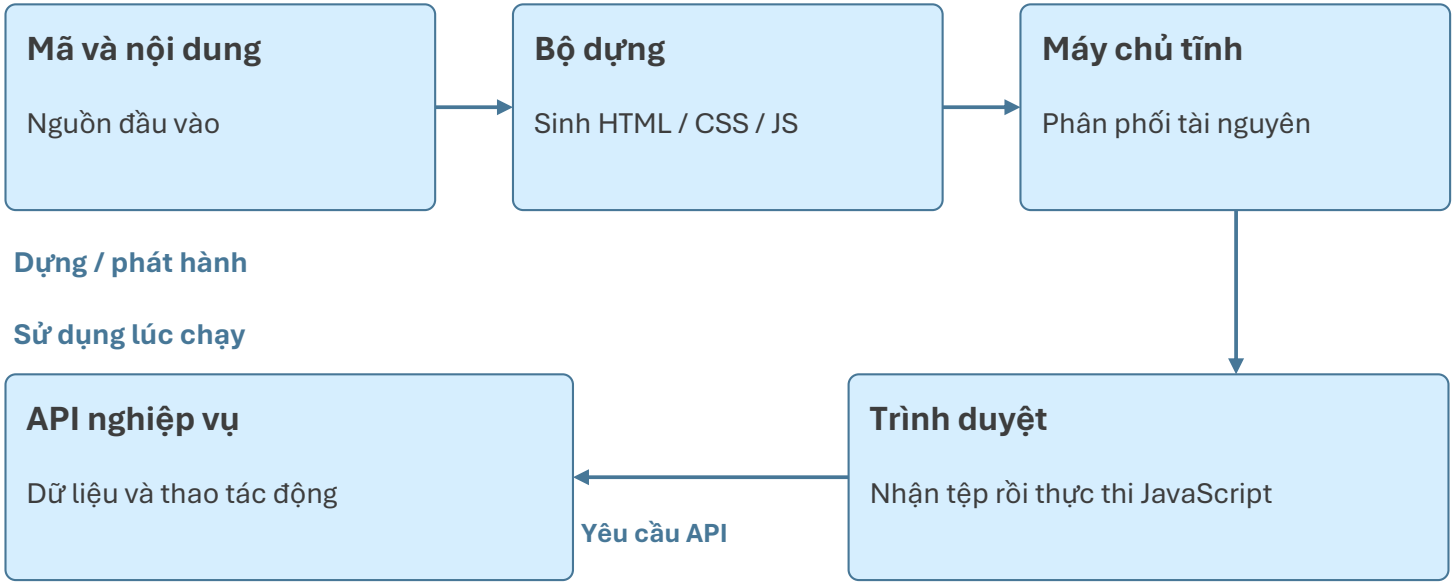
Gọi dịch vụ

Trình duyệt dùng API cho dữ liệu động.

Ranh giới chính: tạo giao diện khi dựng; xử lý nghiệp vụ động qua API.

Nguồn: jamstack.org/what-is-jamstack

Luồng dựng và luồng sử dụng diễn ra khác lúc



Nguồn: Sơ đồ minh họa kiến trúc JAMstack

JAM không giới hạn ứng dụng ở nội dung tĩnh

J — JavaScript

Tương tác và điều hướng
Quản lý phiên
Gọi dịch vụ từ trình duyệt

A — APIs

Xác thực và dữ liệu
Thao tác cần bí mật
Thông báo thời gian thực

M — Markup

HTML dựng sẵn
CSS và tài nguyên
Nền cho trải nghiệm

Frontend tĩnh vẫn có thể cung cấp trải nghiệm động nhờ backend API.

Nguồn: Tổng hợp khái niệm; cấu trúc ScriptHammer

Dựng sẵn đổi chi phí máy chủ thành chi phí tích hợp

Phân phối

Thuận lợi
Tệp dễ cache.

Đánh đổi
Phải quản lý bản cũ.

Nội dung

Thuận lợi
HTML có sẵn.

Đánh đổi
Đổi nội dung cần dựng.

Dịch vụ

Thuận lợi
Tách năng lực backend.

Đánh đổi
Phụ thuộc API và quyền.

Nguồn: Phân tích định tính; không phải số đo hiệu năng

02

Phân tích ScriptHammer

Frontend tĩnh; nghiệp vụ động
Đối chiếu cấu hình và mã nguồn

Nguồn: Mã nguồn ScriptHammer trong workspace

ScriptHammer ghép nội dung với nghiệp vụ động

Giao diện và nội dung

Next.js + React
Blog và tài liệu
Thành phần dùng chung
PWA và hỗ trợ truy cập

Nghiệp vụ người dùng

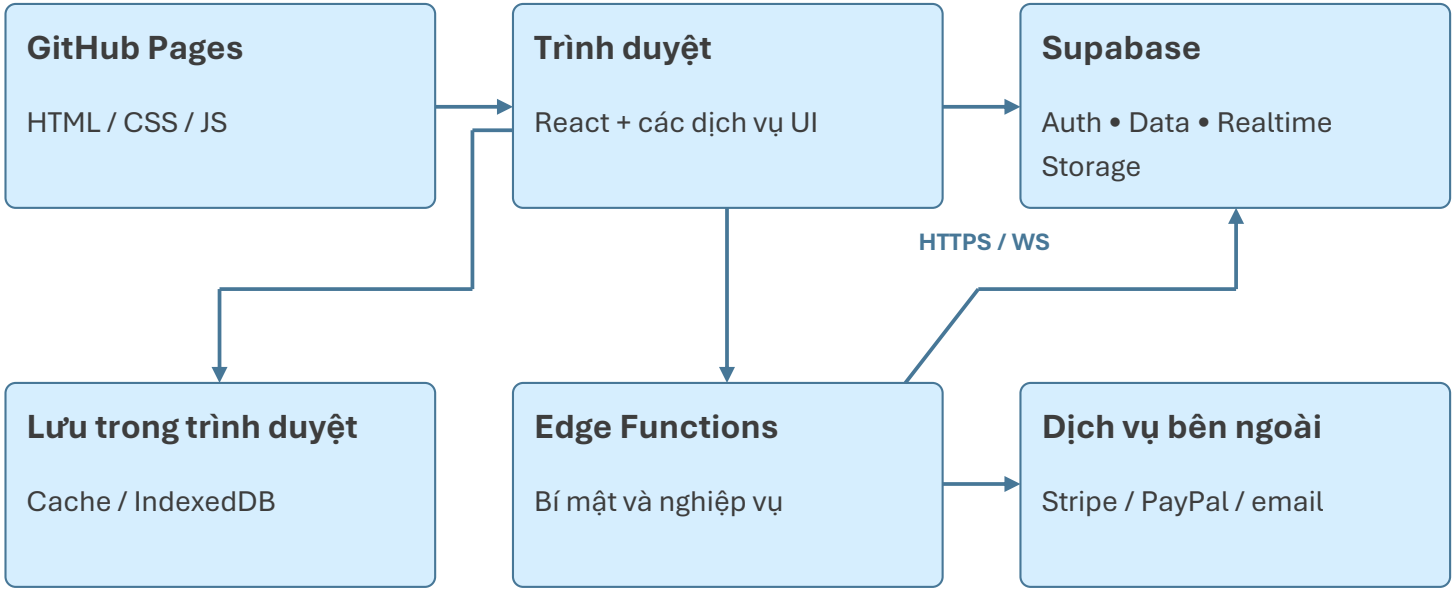
Đăng nhập và hồ sơ
Nhắn tin mã hóa
Thanh toán / thuê bao
Quản trị

Dịch vụ phụ trợ

Supabase Auth / Data
Realtime và Storage
Edge Functions
Stripe, PayPal, email

Nguồn: src/app; src/services; src/lib/payments; public/sw.js

Trình duyệt kết nối frontend tĩnh với backend



Nguồn: next.config.ts; layout.tsx; Supabase client; Edge Functions

Cấu hình và mã xác nhận frontend được xuất tĩnh

Nguyên lý	Bằng chứng trong mã
Dựng trước	output: export; blog đọc JSON khi build
Tuyến sinh sẵn	generateStaticParams cho blog / docs
Phân phối tệp	out/ → artifact → GitHub Pages
Dữ liệu động	Supabase client; gọi Edge Functions

Server Component ở bước build không phải SSR theo từng yêu cầu khi production.

Nguồn: next.config.ts; blog/page.tsx; deploy.yml; stripe.ts

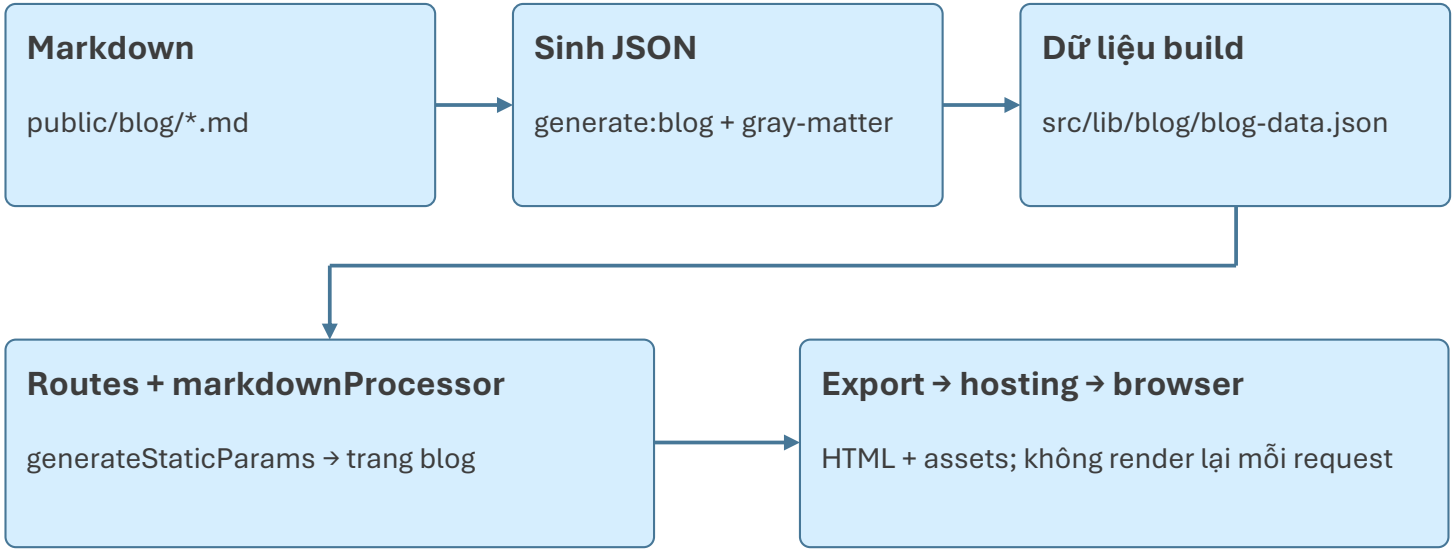
Bốn chế độ khác nhau ở thời điểm tạo HTML

Chế độ	HTML / cập nhật	Trong ScriptHammer
CSR	Browser + API	Auth, messaging, payments
SSR	Server mỗi request	Không có trong static export
SSG	Build → tệp HTML	Blog và routes xuất tĩnh
ISR	Runtime tái tạo trang	Không có trên Pages

Static export + tương tác CSR có thật; SSR và ISR chỉ dùng để so sánh.

Nguồn: next.config.ts; blog/[slug]/page.tsx; Supabase client

Blog đi qua JSON trước khi sinh HTML tĩnh



generate:blog là bước riêng; prebuild không tự sinh lại blog-data.json.

Nguồn: generate-blog-data.js; blog-data.json; blog/[slug]/page.tsx

Nội dung mới cần sinh dữ liệu và phát hành lại

1 / Sửa nguồn

Markdown đổi.

HTML bản A vẫn giữ nội dung đã phát hành.

2 / Dựng bản B

generate:blog → build

Kiểm tra JSON, slug và tệp HTML export.

3 / Phát hành

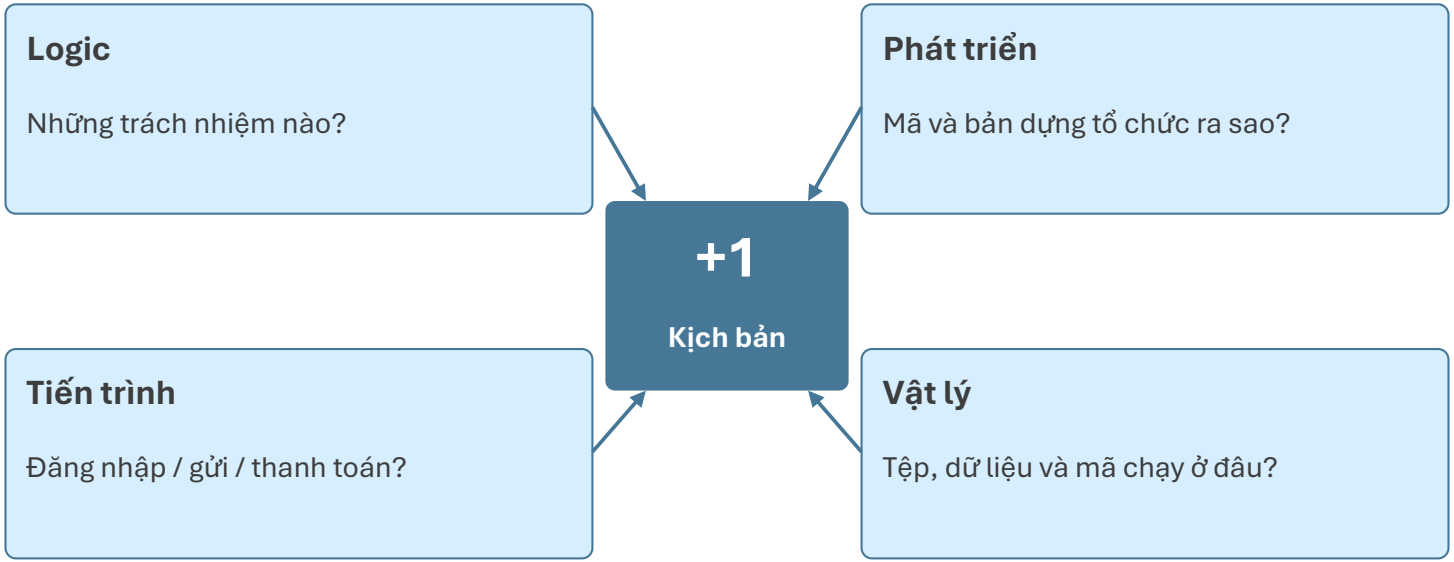
Artifact B → hosting

Đối chiếu cache / SW với một phiên sạch.

API thay dữ liệu động; rebuild thay nội dung đã dựng và cấu hình NEXT_PUBLIC_*.

Nguồn: package.json; deploy.yml; stamp-sw-version.mjs; public/sw.js

Mô hình 4+1 tách năm câu hỏi về hệ thống



Nguồn: Philippe Kruchten, mô hình 4+1, 1995

Góc nhìn logic phân chia giao diện và nghiệp vụ

Giao diện

Routes và components
Context / hooks
Điều hướng, consent
Trạng thái hiển thị

Điều phối nghiệp vụ

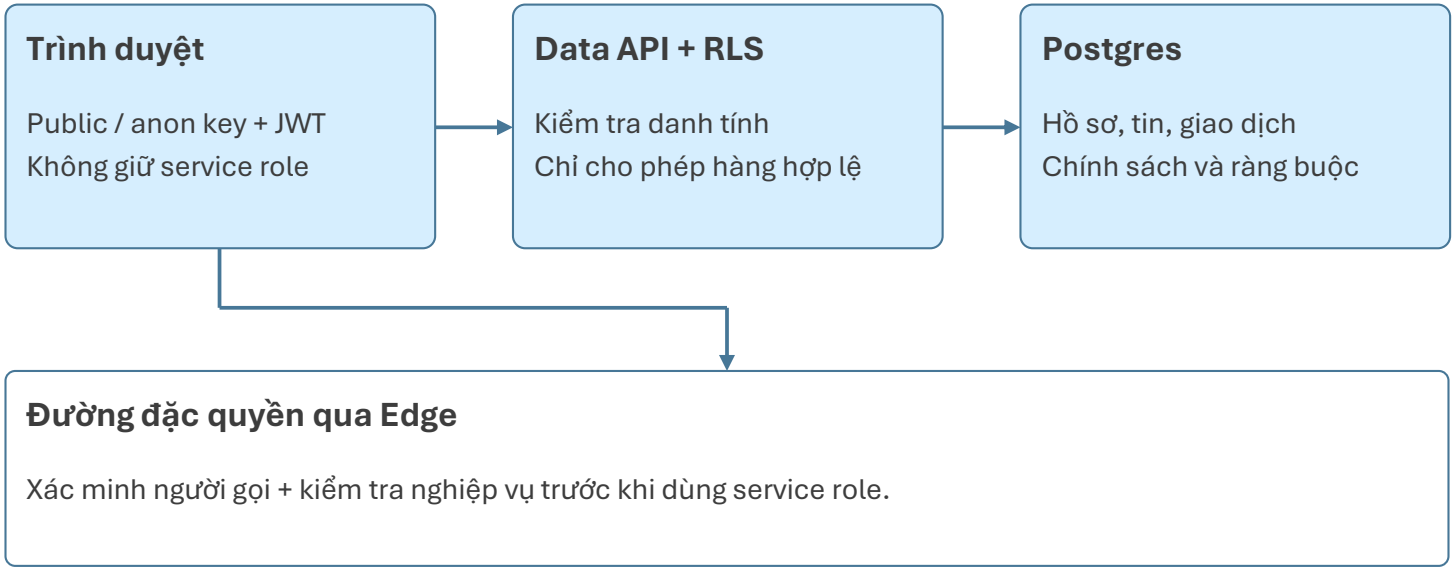
Phiên đăng nhập
Mã hóa và hàng đợi tin
Luồng checkout
Đồng bộ dữ liệu

Năng lực backend

Auth / Postgres / RLS
Realtime / Storage
Edge Functions
Cổng thanh toán

Nguồn: src/app; contexts/AuthContext; services/messaging; lib/payments

Quyền truy cập được thực thi ở backend



Nguồn: ProtectedRoute; SQL migration; create-stripe-checkout

Góc nhìn phát triển có ba nhóm đầu ra riêng

Frontend

src/app, components
contexts, hooks, services
Next.js + TypeScript
Đầu ra: HTML / CSS / JS

Backend

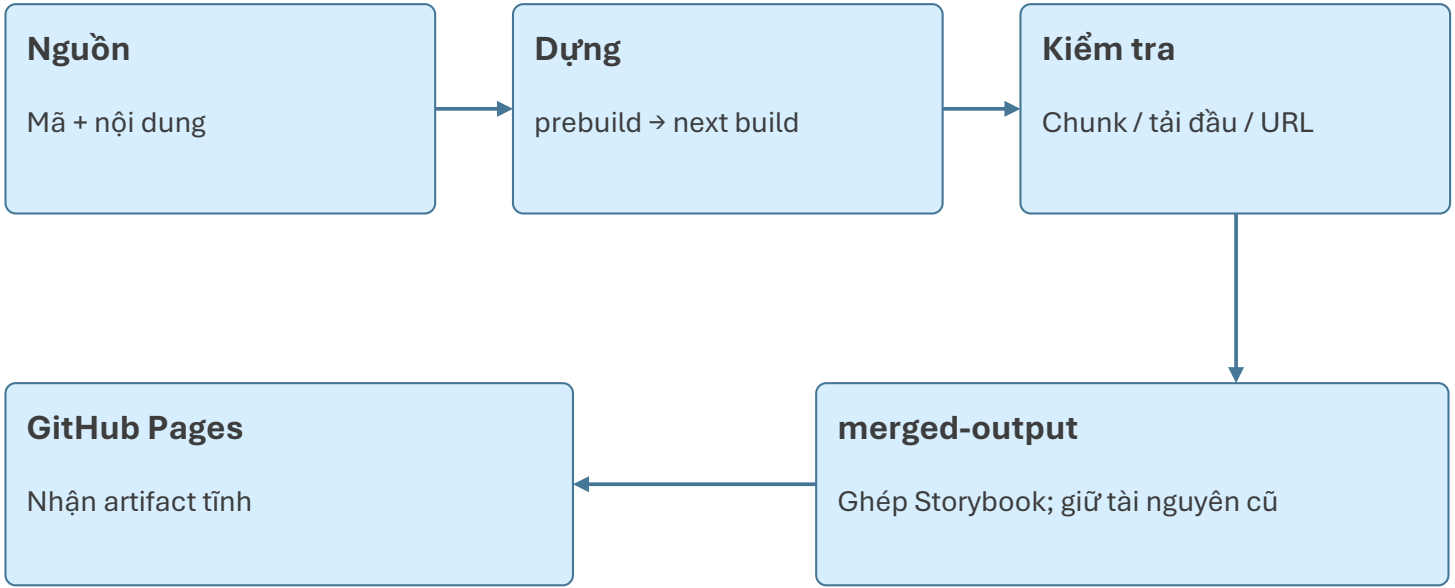
supabase/migrations
supabase/functions
SQL + Deno TypeScript
Đầu ra: schema / API

Công cụ và kiểm tra

scripts, tests
Docker, Storybook
Vitest / Playwright
Workflow GitHub Actions

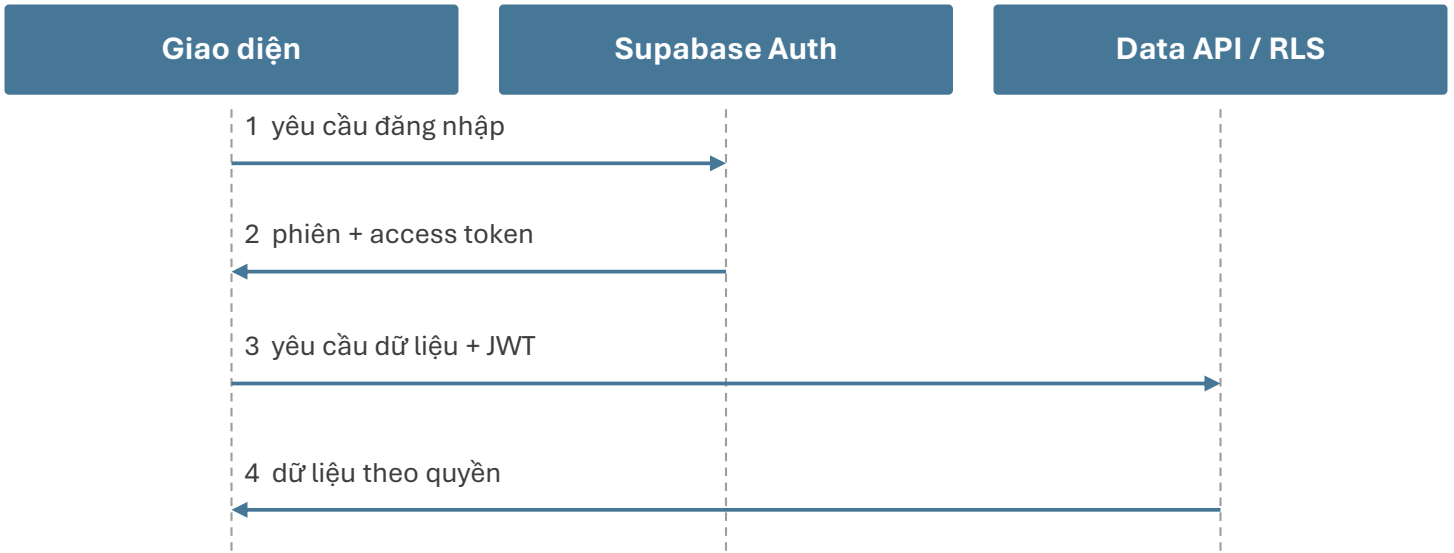
Nguồn: package.json; src/; supabase/; .github/workflows/

Pipeline kiểm tra tệp trước khi phát hành lên Pages



Nguồn: package.json; .github/workflows/deploy.yml

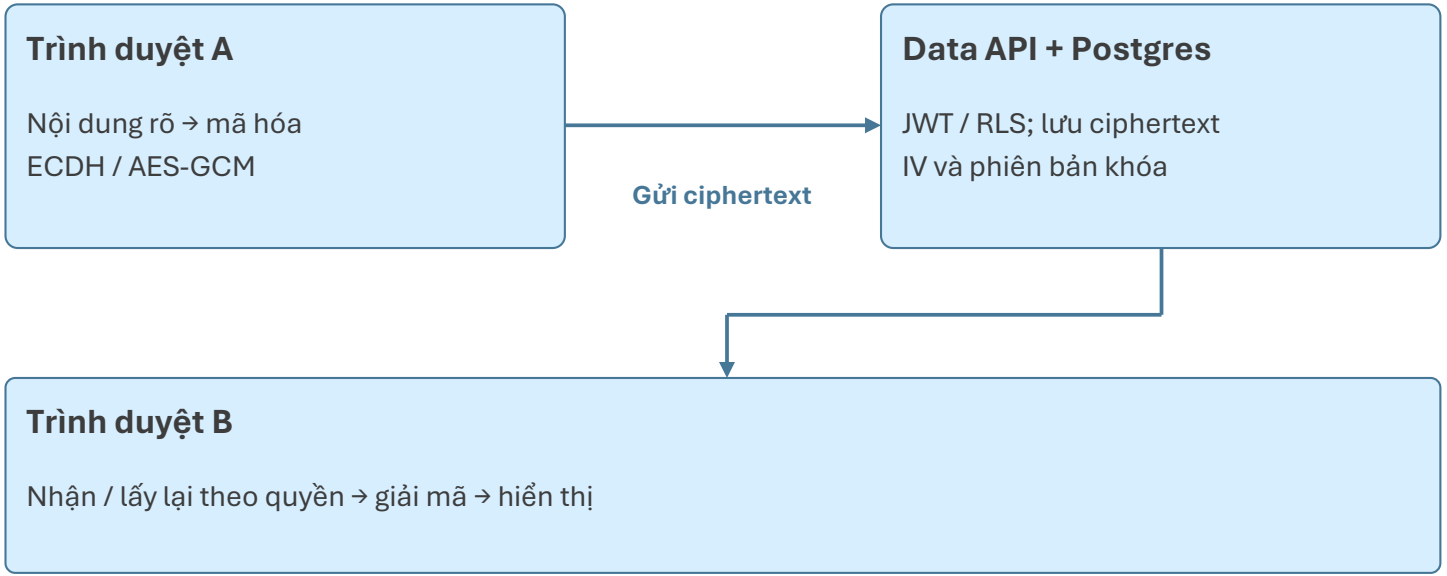
Đăng nhập diễn ra qua API từ trình duyệt



JWT xác định người dùng; RLS quyết định hàng dữ liệu nào được phép truy cập.

Nguồn: AuthContext.tsx; lib/supabase/client.ts; auth/callback

Nội dung tin nhắn được mã hóa trước khi gửi



Nguồn: message-service.ts; encryption.ts; messaging/providers

Ngoại tuyến cần hàng đợi và chính sách đồng bộ

Service worker

Cache trang / tài nguyên
Fallback khi mất mạng
Không chặn mọi request
Không thay backend

IndexedDB

Cache tin đã lấy
Hàng đợi gửi
Trạng thái và khóa
Cần dữ liệu có sẵn

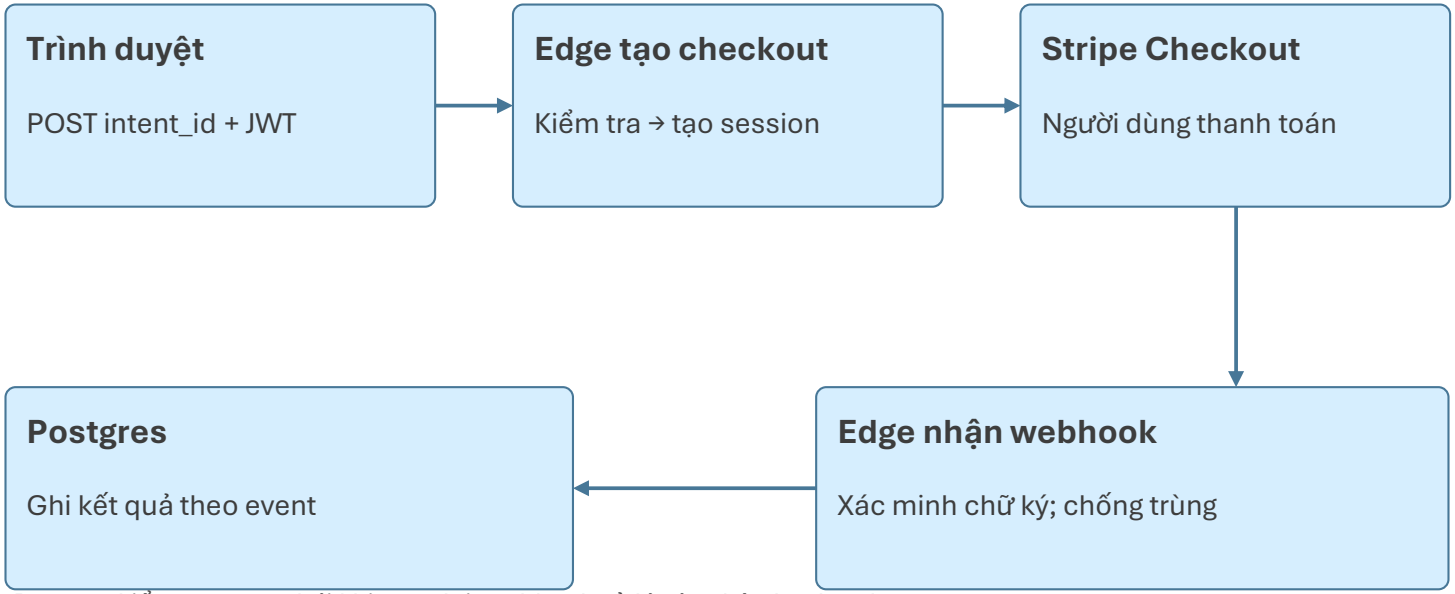
Khi có mạng lại

Thử lại có backoff
UUID chống ghi trùng
Cập nhật trạng thái
Xóa nội dung rõ đã gửi

Tin đang chờ có thể chứa nội dung rõ trong IndexedDB; offline không đồng nghĩa an toàn.

Nguồn: public/sw.js; messaging/database.ts; offline-queue-service.ts

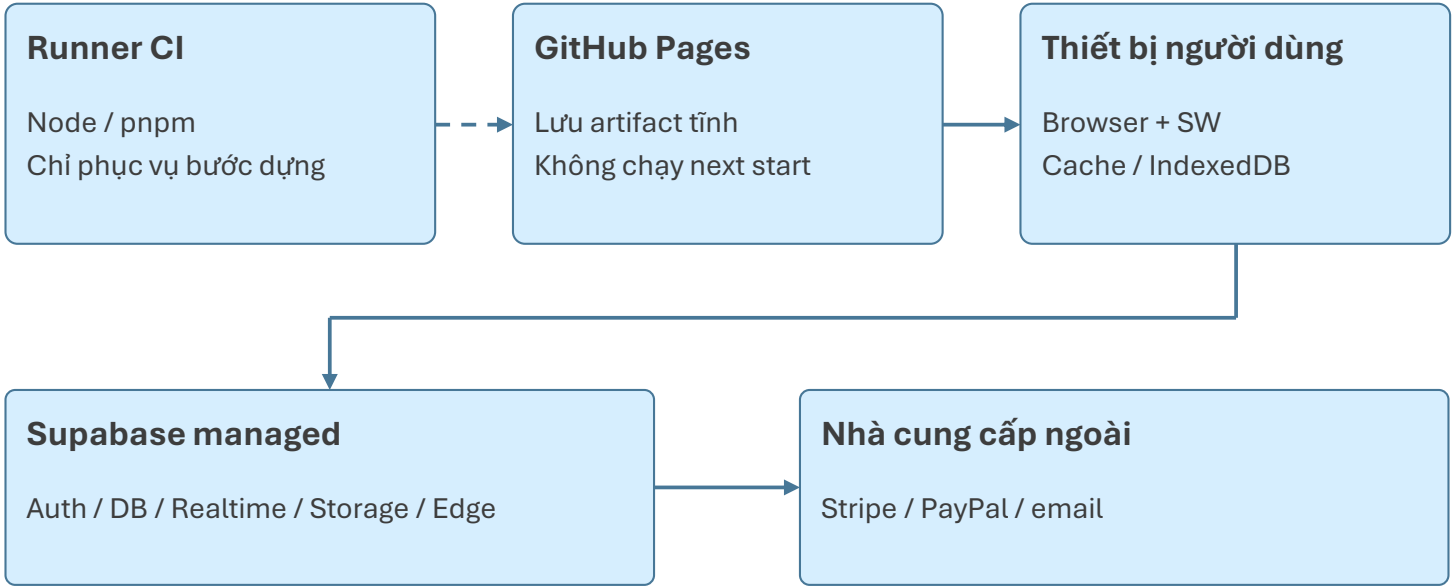
Webhook xác nhận thanh toán; URL trả về chỉ hỗ trợ UI



Browser kiểm tra trạng thái khi quay lại; webhook xử lý xác nhận backend.

Nguồn: payments/stripe.ts; create-stripe-checkout; stripe-webhook

Góc nhìn vật lý tách nơi dựng khỏi nơi thực thi



Nguồn: deploy.yml; next.config.ts; supabase/functions; Docker Compose

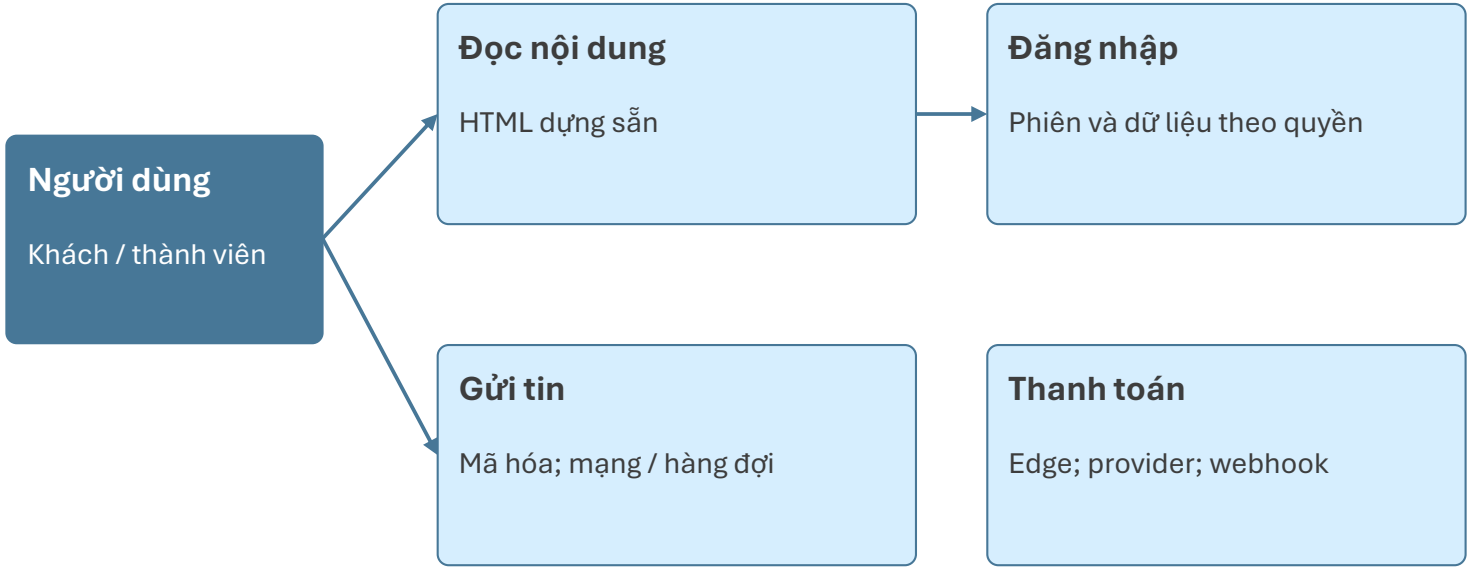
Frontend, schema và secrets có vòng đời khác nhau

Thành phần	Thời điểm thay đổi	Điểm cần quản lý
HTML / JS / CSS	Dựng và deploy Pages	URL, basePath, bản mã
NEXT_PUBLIC_*	Được gắn lúc build	Không phải secrets
Schema và RLS	Áp dụng migration	Tương thích dữ liệu
Edge và secrets	Deploy backend riêng	API và quyền gọi

Đổi biến build không tự cập nhật bundle đã phát hành trên Pages.

Nguồn: deploy.yml; SQL migration; Edge Functions; backend.config.ts

Kịch bản kiểm tra kiến trúc qua hành vi người dùng



Người vận hành: phát hành bản mới khi người dùng vẫn đang giữ HTML cũ.

Nguồn: src/app; AuthContext; messaging; payments; deploy workflow

Kịch bản thanh toán nối đủ bốn góc nhìn

Góc nhìn	Bằng chứng trong kịch bản
Logic	UI điều phối; Edge xác minh; Stripe thu tiền
Phát triển	stripe.ts + Deno function + SQL
Tiến trình	Request → session → webhook → trạng thái
Vật lý	Browser ↔ Supabase ↔ Stripe

Kết quả mong đợi: trạng thái backend đúng, event trùng không ghi kết quả lần nữa.

Nguồn: stripe.ts; create-stripe-checkout; stripe-webhook; migration

Độ bền phụ thuộc cache, API và cơ chế thử lại

Mất kết nối

Có cache → tiếp tục.
Tin có đủ dữ liệu → xếp hàng.

API và dữ liệu mới
vẫn cần mạng.

HTML giữ bản cũ

Giữ chunk cũ theo thời gian.
Kiểm tra artifact.

Cache nhiều lớp
cần phối hợp phiên bản.

Yêu cầu lặp lại

Webhook claim event.
Queue dùng UUID ổn định.

Retry không được
làm lặp tác dụng.

Nguồn: public/sw.js; retain-previous-assets.mjs; queue; webhook-claim

Bảo mật đòi hỏi nhiều ranh giới phối hợp

Danh tính và quyền

UI guard: trải nghiệm.
JWT: danh tính.
RLS: quyền theo hàng.
Edge: kiểm tra nghiệp vụ.

Bí mật backend

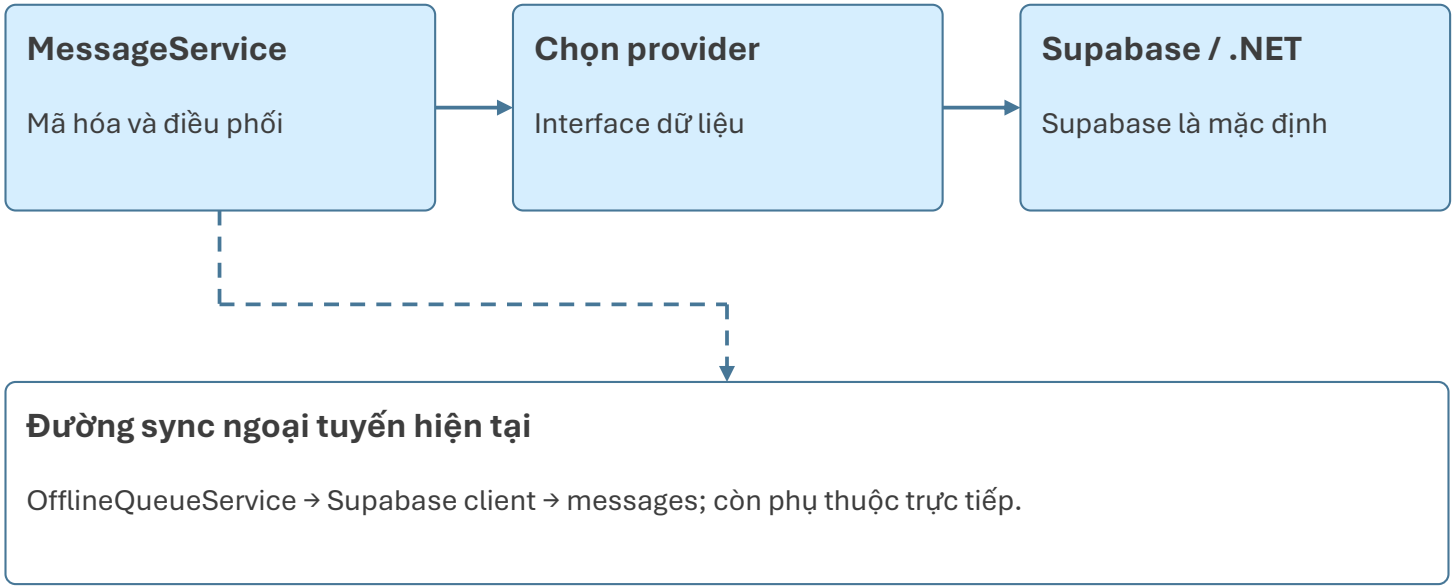
Public key được công khai.
Service role ở backend.
Webhook có chữ ký.
Không tin URL success.

Dữ liệu người dùng

Mã hóa ở browser.
Khóa không extractable.
Còn metadata / pending.
Vẫn phải chống XSS.

Nguồn: ProtectedRoute; SQL RLS; encryption.ts; Edge auth

Đổi provider chưa tách hết phụ thuộc backend



Nguồn: backend.config.ts; providers/index.ts; offline-queue-service.ts

Các quyết định chuyển trách nhiệm sang API và client

Quyết định	Giá trị	Chi phí
Static export	Phân phối tệp đơn giản	Dựng lại và cache
Supabase + RLS	Dữ liệu và quyền có sẵn	Schema / API phụ thuộc
Edge Functions	Bí mật ở backend	Vòng đời triển khai riêng
Hàng đợi cục bộ	Tiếp tục khi mất mạng	Retry, dữ liệu tồn lưu

Nguồn: next.config.ts; SQL policies; browser services; Edge Functions

Cải tiến nên củng cố các ranh giới đã có

1 / Dữ liệu pending

Chính sách lưu nội dung rõ.
Xóa đúng vòng đời.
Kiểm tra logout và retry.
Cân nhắc mã hóa cục bộ.

2 / Provider thống nhất

Send và sync cùng interface.
Giữ kiểm tra quyền.
Chạy conformance chung.
Rà soát realtime / auth.

3 / Hợp đồng phát hành

Kiểm tra artifact cuối.
Thử HTML cũ + chunk mới.
Giữ tương thích schema.
Đối chiếu cấu hình thật.

Nguồn: Đề xuất từ mã hiện tại; không sửa ScriptHammer

Trình diễn cần thử cả frontend tĩnh và backend

1	**Chuẩn bị**	Theo hướng dẫn dự án; xác nhận dịch vụ thử nghiệm.
2	**Dựng tĩnh**	Dùng builder riêng; kiểm tra đầu ra HTML / JS / CSS.
3	**Theo dõi**	Đăng nhập → tin nhắn → mất mạng → đồng bộ lại.
4	**Đối chiếu**	Checkout thử nghiệm; kiểm tra webhook và dữ liệu.

Nguồn: docker-compose.yml; README; deploy workflow

Nguồn: Tổng hợp phân tích ScriptHammer

GIAO DIỆN TĨNH NGHIỆP VỤ ĐỘNG

Ranh giới API và dữ liệu quyết định chất lượng hệ thống

Thảo luận: đổi cấu hình backend có cần phát hành lại frontend?

Tài liệu gốc hỗ trợ các khái niệm kiến trúc

JAMstack — nguyên lý kiến trúc

https://jamstack.org/what-is-jamstack/

Next.js — Static Exports

https://nextjs.org/docs/app/guides/static-exports

Supabase — Row Level Security

https://supabase.com/docs/guides/database/postgres/row-level-security

Supabase — Edge Functions

https://supabase.com/docs/guides/functions

Kruchten — Mô hình 4+1 (1995)

https://www.cs.ubc.ca/~gregor/teaching/papers/4+1view-architecture.pdf

Công cụ: github.com/mpuig/agent-slides

Nguồn: Nguồn chính thức; các liên kết có thể mở từ slide

Bản đồ nguồn phân biệt tài liệu với triển khai thật

Nhận định	Tệp đối chiếu
Static export	next.config.ts; deploy.yml
Nội dung dựng sẵn	app/blog; app/docs; generateStaticParams
Phiên và quyền	AuthContext; supabase/client; SQL RLS
Tin nhắn / ngoại tuyến	message-service; offline-queue-service
Thanh toán / webhook	payments/stripe; supabase/functions
Nhánh backend tùy chọn	backend.config; messaging/providers

Phạm vi: rà soát mã tĩnh; số liệu trong tài liệu cũ được đối chiếu lại.

Nguồn: ScriptHammer; commit a2dd31ae25ea30c678d990432c7e2b60b4faeda7

CẢM ƠN!

Câu hỏi và thảo luận

Khi API mất kết nối, phần nào
của ScriptHammer vẫn tiếp tục hoạt động?

GHI CÔNG: Mẫu trình chiếu do Slidesgo, biểu tượng: Flaticon, đồ họa và hình ảnh: Freepik; minh họa: Stories

Nguồn: Mẫu Slidesgo; mã nguồn ScriptHammer; agent-slides